

1) Unsupervised learning is when our data is **not explicitly labeled**, and we ask the algorithm to learn something meaningful on its own without labels.

2) The 2 types of unsupervised learning we will explore today are: **K-Means, PCA (principal component analysis)**

3) What is a centroid? Take some time on this question after reading under the K-Means block in the ipynb. Draw some of your understanding out on the board! Include some formulas if you can.

A centroid is a mean/average point

```
idx=findClosestCentroids(X, centroids);
```

```
centroids = computeMeans (X, idx, K);
```

4) Under line 7, the blue points represent **data points** and the stars represent **centroids**.

5) Write the formula for finding the closest centroids on the board, and then summarize it using your own words here.

Finding the distance by minimizing the square magnitude of the position of $x(i)$ minus the position of the centroid

6) Read the code in line 8. Name the type of python object each of these are:

centroids -> float

X-> array

np.square -> method

min_distance -> integer/float

idx -> float

7) What is the purpose of line 9?

It outputs the closest centroid for the first three datapoints

8) What is the difference between the first plot under line 7 versus the one in line 10?

The first plot only has 1 cluster while the second one has three plots

9) On the board, write the equation under 'Computing Centroid Means' and label each of the variables. On your canvas submission, write 2 sentences on what this equation represents.

The equation is computing a weighted average of k cluster by adding up all the $x(i)$ s in the cluster (assigned to its centroid), and dividing by the set.

10) Challenge: What is the purpose of line 17-24?

Runs the KMeans loop for a fixed number of iterations and tracks centroids

11) Line 25 onwards shows you how to do exactly what the above lines did with hard code. That means that scikit learn has an inbuilt function for all the things we just hand-coded! Yay! Write the code that helps you do what we did faster here:

```
from sklearn.cluster import KMeans
```

```
km1= KMeans (3)
```

```
km1.fit (X1)
```

12) Look up the parameters for Kmeans and explain them here!

n_clusters: # of clusters to form

init: method for initialization

n_init: # of times the k-means algorithm is run with different centroid seeds

max_iter: max number of iterations

tol: relative tolerance with regards to Frobenius norm of the difference in the cluster centers of two consecutive iterations to declare convergence.

verbose: verbosity mode

random_state: determines random number generation

copy_x: if to make a copy of x

algorithm: kmeans algorithm to use

13) Check out the image compression example from lines 27-36, and write at least 3 questions on the board that you have about it (if you don't have any questions, write 3 'takeaways'.

14) It's PCA time. What type of object is X3 in line 39?

X3 is an array/matrix with 50 rows and 2 columns

15) What type of normalization are they doing for this data?

This is a z-score normalization/standardization

16) Vectors are kinda hard. Find a resource online to help you understand #3-- I'm here to help! Insert the resource here.

<https://www.youtube.com/watch?v=PFDu9oVAE-g>

17) In line 42, what is the purpose of this function, and what type of objects is the function returning?

The function does all three steps from the previous image, and performs the PCA. The function returns 6 objects:

Mean - float

Std - float

X_norm- float

U- array (list of lists)

S - list

V - array (list of lists)

18) Don't worry about all the particularities of line 47, data visualization is it's own class and specialty, but check out the graph. Read the paragraph just below and explain it in a sentence in normal-people speak here. I would ignore the paragraph after that unless you're feeling brave.

Since most of the data's spread is following the line $y=x$, PCA lets us change the 2D data to just 1 D by projecting every point onto that black line.

19) Skip ahead to the graph under line 54. Write a sentence about what this is doing for me.

Each blue dot is the original 2D data and now they have been aligned to the diagonal line. The dashed lines represent the small amount of information that gets lost in that compression.

20) Lines 55 do the thing that happened in the above lines for PCA with an in-built function in scikit learn. Insert the code here:

```
from sklearn.decomposition import PCA
```

```
pca= PCA (n_components= 2)
```

```
pca_result= pca.fit_transform(X3)
```

21) In the code you just wrote above, where did X3 even come from? What is it? And what is n_components anyway?

X3 was an array defined earlier that contains all the data. n_components tells PCA how many dimensions you want to keep after compression. Here n_components=2 is actually the same dimensionality as X3, so its actually not doing anything.

22) Somewhere before the graph are a few sentences that I would like you to comprehend:

```
pca_result: (50, 2)
```

```
Variation per principal component: [0.87062385 0.12937615]
```

The above ratio is how much of the variance or variability in our data is captured by the two principal components.

You may have to look this up, but what does the (50, 2) mean? And in normal people speak, what does the output for Variation per principal component mean?

(50, 2) is the shape of the output array. It means that the first principal component (the black diagonal line) captures 87% of all the variability in the data, while the second component only captures the remaining 13%.